

Architecture Overview

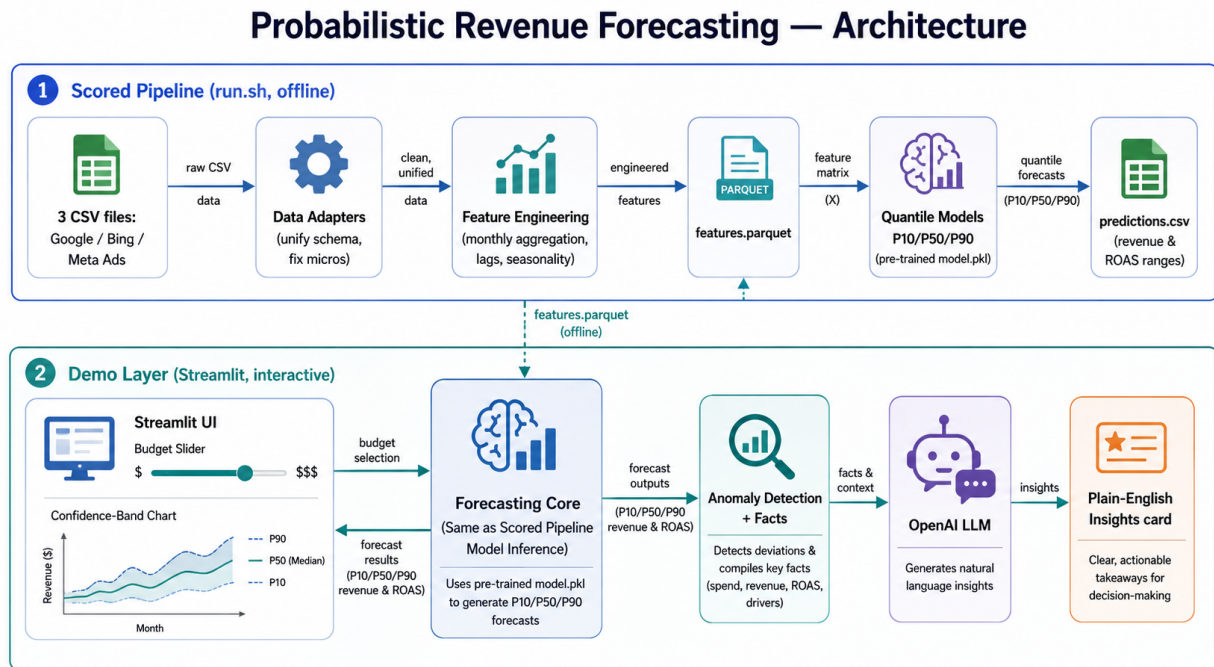


Figure 1: System architecture

Stack

Layer	Technology
Language	Python 3.11
Data / compute	pandas, numpy, pyarrow
Modeling	scikit-learn (GradientBoostingRegressor, quantile loss), joblib
Frontend / demo	Streamlit + Plotly
LLM integration	OpenAI API (via <code>openai</code>), config via <code>python-dotenv</code>

Two dependency sets are kept separate on purpose: - `requirements.txt` — minimal, for the **scored** pipeline (no internet, no LLM). - `requirements-app.txt` — adds Streamlit / Plotly / OpenAI for the **demo**.

Two execution paths

```

===== SCORED PATH (run.sh) =====
data/*.csv -> generate_features.py -> features.parquet -> predict.py -> output/predictions.csv
              (data_loader + features)                (forecast + model.pkl)

===== DEMO PATH (streamlit) =====
data/*.csv -> app.py -> build_panel -> run_forecast(budget) -> charts + tables
              |
              +-> insights.py -> OpenAI API -> insight card
  
```

The scored path is fully offline and deterministic. The demo path reuses the exact same forecasting core and only adds interactivity + the LLM layer.

Module map (src/)

Module	Responsibility
config.py	canonical schema, campaign-type normalization
data_loader.py	per-platform adapters -> unified long table
features.py	monthly aggregation, lag/rolling/seasonality, partial-month filter
generate_features.py	entry: <code>data/</code> -> <code>features.parquet</code>
model.py	design matrix + three quantile regressors
train.py	offline backtest + train + save <code>pickle/model.pkl</code>
forecast.py	recursive roll-forward, budget simulation, horizon roll-ups
predict.py	entry: features + model -> <code>predictions.csv</code>
insights.py	facts + anomaly rules + LLM explanation (with fallback)
app.py (root)	Streamlit UI

Forecasting pipeline (data flow)

1. **Ingest & unify** — adapters normalize the three platforms into one table.
2. **Aggregate & feature** — daily -> monthly; add lag/rolling/seasonality clues.
3. **Predict** — three quantile models -> revenue ranges; ROAS derived.
4. **Roll forward & up** — recursive months -> 30/60/90 days; channel/type/total.
5. **Explain** (demo only) — grounded facts + anomalies -> LLM insight.

LLM integration workflow

```
forecast + history -> compute_facts() ----+
                                         +-> prompt (facts only) -> OpenAI -> summary
panel -> detect_anomalies() -----+      |
                                         (no key / error) -----+> rule-based fallback
```

The LLM never sees raw data and is constrained to the computed facts, which keeps insights grounded and avoids hallucinated numbers.